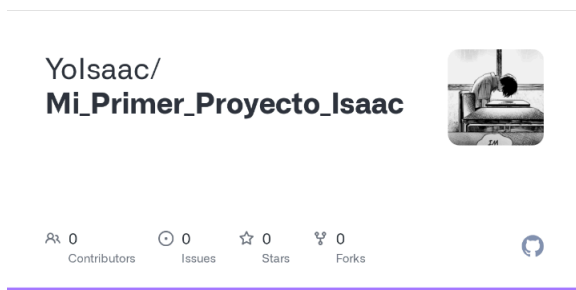


Desarrollo de aplicaciones móviles



Actividad 2



https://github.com/Yolsaac/Mi_Primer_Proyecto_Isaac.git

De: Isaac Alejandro Isaias Betance

Descripción

En esta actividad, el aprendedor asumirá el rol de un ingeniero de software encargado de modernizar el sistema de administración de estacionamientos de la empresa ParkSmart. Para ello, analizará los desafíos del sistema actual, como largas filas y poca flexibilidad en los pagos, y propondrá mejoras mediante tecnologías avanzadas. Evaluará distintas metodologías de desarrollo de software, para seleccionar la más adecuada según los requerimientos del proyecto. Además, identificará y documentará especificaciones clave en cada fase del ciclo de vida del software (SDLC), asegurando la escalabilidad y eficiencia del sistema.

Comprender los fundamentos de la ingeniería de software, su impacto en el desarrollo de aplicaciones y las implicaciones éticas en su uso. Asimismo, se espera que los estudiantes analicen las diferentes fases del ciclo de vida del software y los modelos de desarrollo más utilizados para gestionar proyectos de software de manera eficiente.

Analiza el siguiente caso:

La empresa ParkSmart ha enfrentado problemas en la construcción de proyectos debido a una mala planificación y la selección inadecuada de modelos de desarrollo, generando retrasos en la entrega, sobrecostos y errores críticos en los sistemas implementados.

Además, los nuevos desafíos como la integración de inteligencia artificial (IA) y la falta de experiencia, así como la ausencia de estrategias adecuadas para gestionar la seguridad y la ética, ha generado preocupaciones adicionales.

Actúa como consultor de ingeniería de software para evaluar la situación y proponer estrategias de mejora. Tu tarea es analizar y evaluar los diferentes modelos de desarrollo para determinar cuál se adapta mejor a las necesidades de la empresa. Además, deberás identificar casos similares en los que una mala gestión del ciclo de vida del software haya causado fallos, presenta un plan de mejora basado en buenas prácticas, considerando los retos como el uso de inteligencia artificial, automatización y ciberseguridad en el desarrollo de sus proyectos.



Instrucciones:

Investiga empresas en las que una mala gestión del ciclo de vida del software haya provocado fracasos en sus proyectos. Identifica la causa y propón una estrategia que lo hubiera evitado .

Respuesta:

Estuve investigando un par de casos reales donde por no llevar bien las fases del ciclo de vida del software (SDLC) la terminaron regando feo , aquí están los dos ejemplos con su análisis :

-

El Sistema Automatizado de Equipaje del Aeropuerto de Denver (DIA) :

-.

¿Que pasó? : Por allá de 1994 quisieron hacer un sistema gigante para mover maletas en todo el aeropuerto de forma automática . El problema fue que eligieron un enfoque súper rígido tipo Cascada y se brincarón la parte de hacer pruebas integrales por fases . Como no probaron todo bien junto , los carritos tiraban las maletas , se atoraban y rompían las cosas . Esto hizo que el

aeropuerto abriera con mas de 16 meses de retraso y perdieran como 560 millones de dolares .

-.

Relación con el SDLC : Faltó un análisis real de integración y se saltaron las pruebas continuas (Testing) antes de querer desplegar todo en producción .

-.

Estrategia para evitarlo : Debieron usar un modelo Incremental . Ir probando primero una sola terminal o una sola banda de maletas , corregir los errores sobre la marcha y luego ya ir escalando al resto del aeropuerto .

-

El software MCAS del Boeing 737 MAX :

-.

¿Que pasó? : Miren en este caso metieron un software nuevo llamado MCAS para estabilizar el avión en el aire pero falló feo el análisis de riesgo y la seguridad . El software dependía de un solo sensor de ángulo y si ese sensor fallaba el sistema obligaba al avión a picar hacia abajo . Además por la prisa de sacarlo al mercado no documentaron bien el sistema ni capacitaron a los pilotos en las pruebas de uso . Lamentablemente esto causó dos accidentes fatales .

-.

Relación con el SDLC : Fallaron cañón en la fase de Análisis de Requerimientos de Seguridad y en las Pruebas de Redundancia .

-.

Estrategia para evitarlo : Aplicar un modelo Espiral desde el inicio . Como este modelo se enfoca 100% en evaluar riesgos en cada vuelta , hubieran detectado a tiempo el riesgo de depender de un solo sensor y habrían agregado sensores de respaldo antes de autorizar el vuelo .

Compara las ventajas y desventajas del modelo de desarrollo de software que usarás en tu propuesta de solución. Los modelos de trabajo son cascada, espiral, incremental y evolutivo.



Respuesta:

Hice esta tabla comparativa para revisar cómo está el rollo con los cuatro modelos y ver cuál nos conviene más para arreglar el broncón de ParkSmart :

Modelo	Ventajas	Desventajas	Aplicación en proyectos reales
Cascada	Es muy ordenado , sabes qué entregables hay en cada etapa y es fácil de entender si el	Cero flexible . Si te equivocas en los requisitos del principio cambiar algo al final te sale	Proyectos pequeños con requerimientos que jamás cambian , tipo un sistema de nómina súper

	equipo va empezando .	carísimo o tienes que rehacer todo .	básico .
Espiral	Se enfoca muchísimo en la evaluación de riesgos . Es perfecto cuando metes cosas raras como Inteligencia Artificial o ciberseguridad .	Es bastante caro y necesitas gente muy experta para estar evaluando los riesgos en cada iteración .	Sistemas críticos donde si falla algo se pierde mucho dinero o vidas (como la aviación o bancos) .
Incremental	Vas entregando la app por partes (módulos) . El cliente ve avances rápido y puedes corregir detalles sin detener todo .	Si no diseñas bien la arquitectura inicial , juntar los módulos nuevos con los viejos se puede volver un spaghetti .	Apps móviles que van creciendo poco a poco , como las apps de delivery o de estacionamientos .
Evolutivo	Desarrollas prototipos rápidos para que el usuario los pruebe y diga qué le parece . Sirve para entender bien lo que quieren .	Si no le pones un alto al cliente te va a pedir cambios infinitos y el proyecto nunca se termina .	Aplicaciones donde la interfaz (UX/UI) es lo mas importante y no está claro cómo interactuará la gente .

Propuesta para ParkSmart :

Para ParkSmart la combinación ganadora es usar un enfoque Incremental combinado con Espiral . Con la parte Incremental sacamos primero lo básico que es ver cajones libres y pagar , y luego en los siguientes incrementos le metemos lo de IA . Y usamos la parte de Espiral para ir evaluando los riesgos de ciberseguridad al momento de procesar los pagos y conectar las cámaras .

Investiga a qué retos éticos se enfrenta la empresa ParkSmart en sus soluciones propuestas.

Respuesta:

Al meter tecnología como cámaras con IA y cobros automáticos en ParkSmart me di cuenta que salen un par de dilemas éticos que hay que cuidar bastante :

- Privacidad y manejo de datos personales (Rastreo de placas y cámaras) :

-.

El reto : Si ParkSmart pone cámaras con IA para leer las placas de los carros y guardar a qué hora entran y salen , está recopilando datos de ubicación y hábitos de los usuarios . Si esos datos se filtran o los venden a terceros , se rompe feo la privacidad de la gente .

-.

Solución / Buena práctica : Aplicar la regla de "Privacidad desde el Diseño" . Esto significa encriptar las placas en la base de datos (usar hashing) , no guardar el historial de rutas por más de 24 horas y avisar a los clientes en los términos y condiciones cómo se maneja su información .

-

Sesgos y cobros injustos en la IA (Precios dinámicos) :

-.

El reto : Si dejamos que un algoritmo de IA cambie las tarifas del estacionamiento según la demanda , podría pasar que en una emergencia o zona transitada suba los costos exageradamente afectando a personas con menos recursos .

-.

Solución / Buena práctica : Definir límites máximos de tarifa fijados por humanos (un tope) para que la IA nunca cobre de más por puro capricho del algoritmo , manteniendo la transparencia de cómo se calcula el precio .

Elabora un mapa conceptual que represente el ciclo de vida del software y su relación con los modelos de desarrollo:

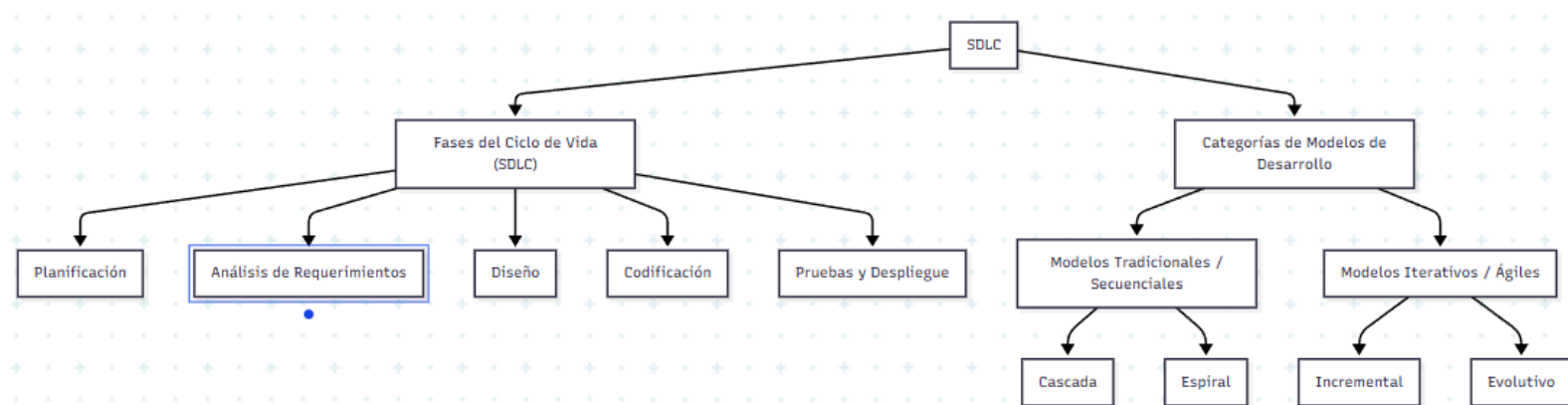
Puede incluirse el uso de herramientas como Lucidchart o Draw.io para la representación visual del ciclo de vida del software.

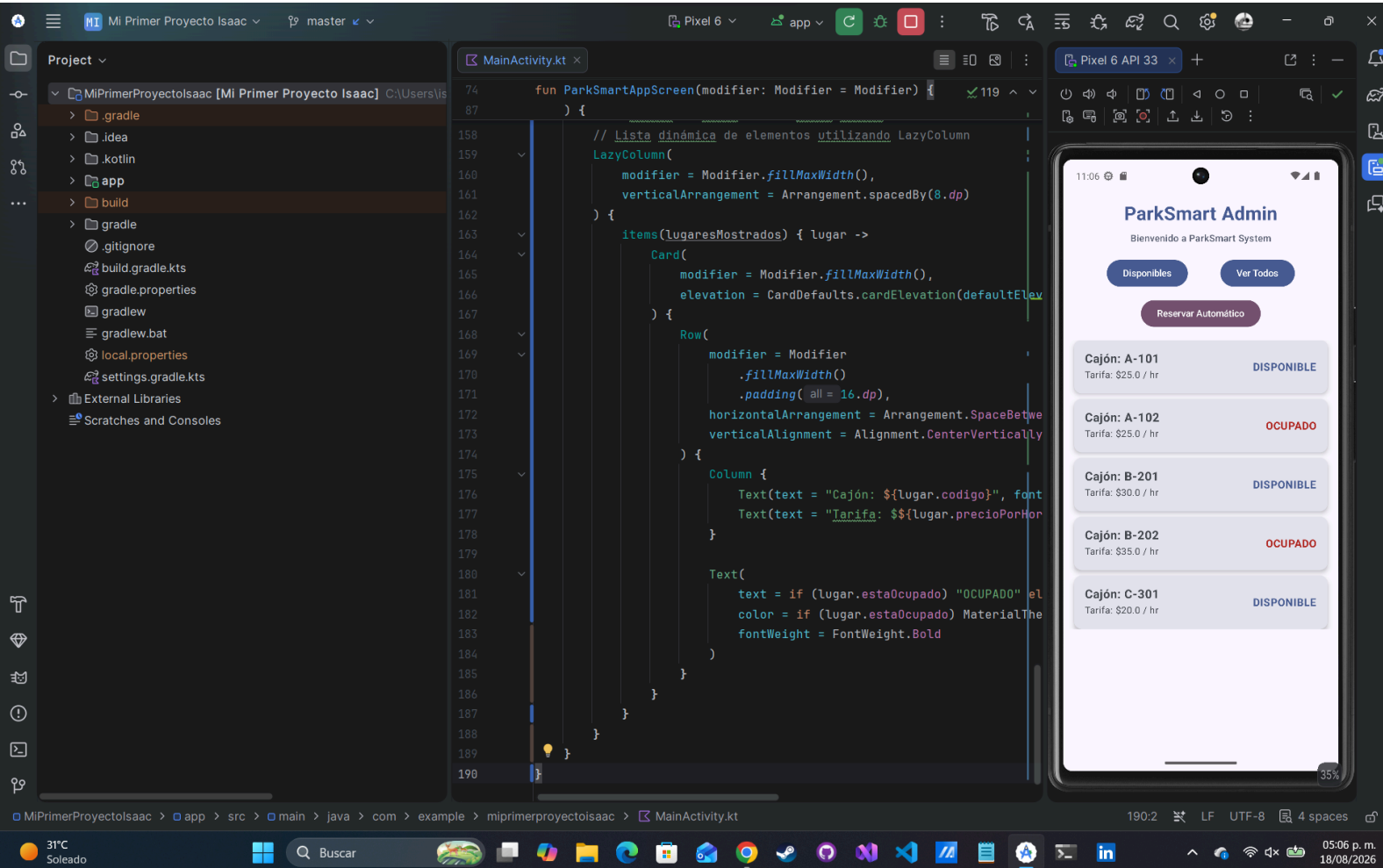
Se recomienda revisar los cursos de LinkedIn Learning relacionados con la gestión de proyectos de software.

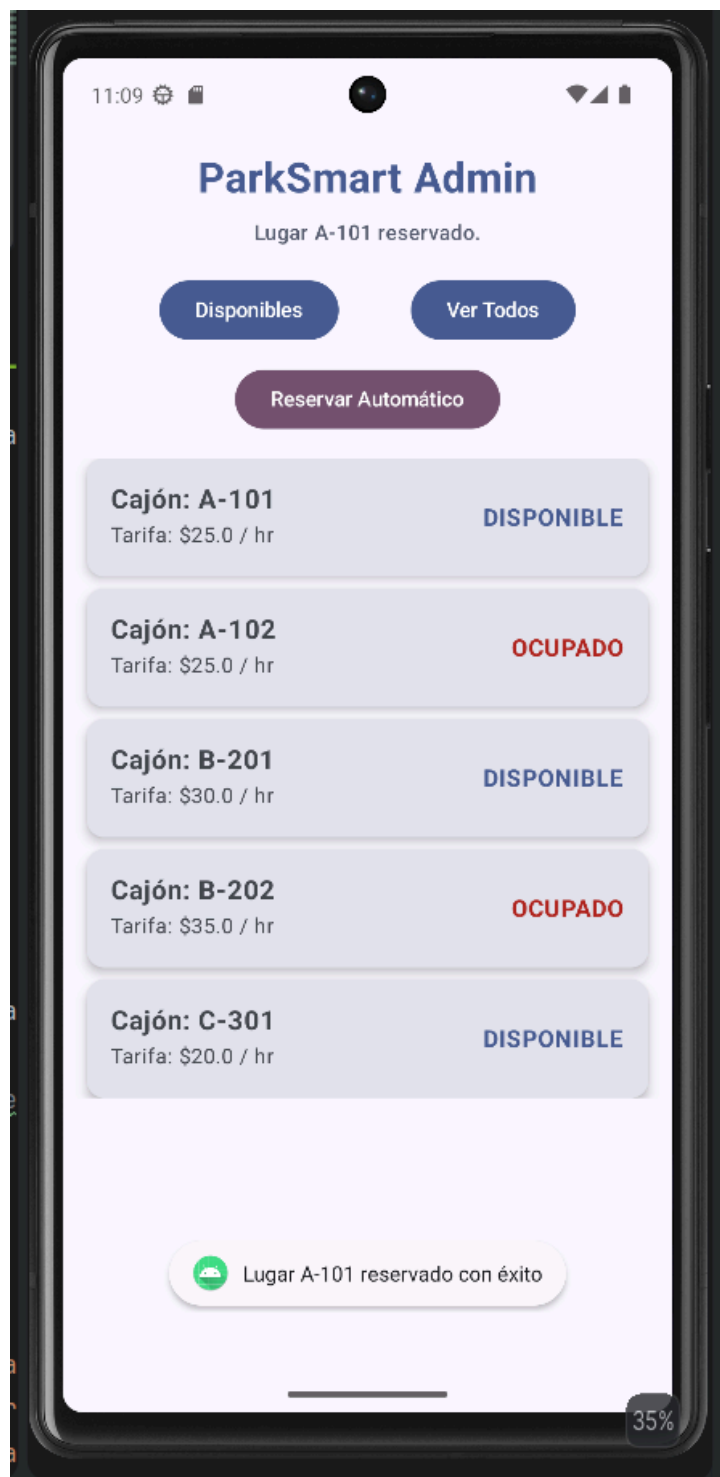


Respuesta:

Aquí dejo la estructura de cómo se conecta todo el ciclo de vida del software (SDLC) con los modelos de desarrollo que estuvimos analizando :







Código en Kotlin utilizado para el prototipo funcional:

```
package com.example.miprimerproyectoisaac
```

```
import android.os.Bundle
import android.widget.Toast
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
```

```
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.miprimerproyectoisaac.ui.theme.MiPrimerProyectoIsaacTheme
```

```
// Modelo de datos para la app de ParkSmart
```

```
data class LugarEstacionamiento(
    val id: Int,
    val codigo: String,
    val estaOcupado: Boolean,
    val precioPorHora: Double
)
```

```
// Excepción personalizada para simular manejo de errores
```

```
class EstacionamientoLlenoException(mensaje: String) : Exception(mensaje)
```

```
// Clase encargada de administrar la lógica de los lugares (Colecciones y Funciones de Orden Superior)
```

```
class ParkSmartManager {
    // Uso de Colección (MutableList)
    private val listaLugares = mutableListOf(
        LugarEstacionamiento(1, "A-101", false, 25.0),
        LugarEstacionamiento(2, "A-102", true, 25.0),
        LugarEstacionamiento(3, "B-201", false, 30.0),
        LugarEstacionamiento(4, "B-202", true, 35.0),
        LugarEstacionamiento(5, "C-301", false, 20.0)
    )
```

```
fun obtenerTodos(): List<LugarEstacionamiento> = listaLugares
```

```
// //Inicia el rollo con el pollo inicio .
```

```
// Esta es una función de Orden Superior porque recibe un predicado como parámetro para filtrar la lista
```

```
fun filtrarLugares(filtro: (LugarEstacionamiento) -> Boolean): List<LugarEstacionamiento> {
    return listaLugares.filter(filtro)
}
```

```
// //Inicia el rollo con el pollo inicio .
```

```
// Método que incluye manejo de excepciones simulado
```

```
fun reservarLugarDisponible(): LugarEstacionamiento {
    val disponible = listaLugares.firstOrNull { !it.estaOcupado }
    return disponible ?: throw EstacionamientoLlenoException("¡Ups! No hay lugares disponibles en ParkSmart por el momento.")
}
```

```
class MainActivity : ComponentActivity() {
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
```

```
        super.onCreate(savedInstanceState)
```

```
        enableEdgeToEdge()
```

```
        setContent {
```

```
            MiPrimerProyectoIsaacTheme {
```

```
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
```

```
                    ParkSmartAppScreen(modifier = Modifier.padding(innerPadding))
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

```
@Composable
```

```
fun ParkSmartAppScreen(modifier: Modifier = Modifier) {
```

```
    val context = LocalContext.current
```

```
    val manager = remember { ParkSmartManager() }
```

```
// Estados para la interfaz
```

```
var lugaresMostrados by remember { mutableStateOf(manager.obtenerTodos()) }
```

```
var mensajeEstado by remember { mutableStateOf("Bienvenido a ParkSmart System") }
```

```
Column{
```



```

        modifier = modifier
            .fillMaxSize()
            .padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        // //Inicia el rollo con el pollo inicio .
        // Encabezado principal
        Text(
            text = "ParkSmart Admin",
            fontSize = 28.sp,
            fontWeight = FontWeight.Bold,
            color = MaterialTheme.colorScheme.primary
        )
    }

```

```

Spacer(modifier = Modifier.height(8.dp))

```

```

Text(
    text = mensajeEstado,
    fontSize = 14.sp,
    fontWeight = FontWeight.Medium,
    color = MaterialTheme.colorScheme.secondary
)

```

```

Spacer(modifier = Modifier.height(16.dp))

```

```

// //Inicia el rollo con el pollo inicio .
// Botones de filtro que utilizan la Función de Orden Superior
Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.SpaceEvenly
) {
    Button(onClick = {
        // Filtra solo disponibles usando la función de orden superior
        lugaresMostrados = manager.filtrarLugares { !it.estaOcupado }
        mensajeEstado = "Mostrando solo lugares disponibles"
    }) {
        Text("Disponibles")
    }
}

```

```

Button(onClick = {
    // Muestra todos
    lugaresMostrados = manager.obtenerTodos()
    mensajeEstado = "Mostrando todos los lugares"
}) {
    Text("Ver Todos")
}
}

```

```

Spacer(modifier = Modifier.height(12.dp))

```

```

// //Inicia el rollo con el pollo inicio .
// Botón para simular la reserva con manejo de excepciones
Button(
    colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.tertiary),
    onClick = {
        try {
            val reservado = manager.reservarLugarDisponible()
            Toast.makeText(context, "Lugar ${reservado.codigo} reservado con éxito", Toast.LENGTH_SHORT).show()
            mensajeEstado = "Lugar ${reservado.codigo} reservado."
        } catch (e: EstacionamientoLlenoException) {
            // Captura la excepción personalizada
            Toast.makeText(context, e.message, Toast.LENGTH_LONG).show()
            mensajeEstado = "Error: Sin lugares disponibles"
        } catch (e: Exception) {
            // Captura cualquier otro error no esperado
            Toast.makeText(context, "Error desconocido: ${e.localizedMessage}", Toast.LENGTH_SHORT).show()
        }
    }
) {
}
}

```

```
Text("Reservar Automático")
}
```

```
Spacer(modifier = Modifier.height(16.dp))
```

```
// // Inicia el rollo con el pollo inicio .
// Lista dinámica de elementos utilizando LazyColumn
LazyColumn(
    modifier = Modifier.fillMaxWidth(),
    verticalArrangement = Arrangement.spacedBy(8.dp)
) {
    items(lugaresMostrados) { lugar ->
        Card(
            modifier = Modifier.fillMaxWidth(),
            elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)
        ) {
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(16.dp),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Column {
                    Text(text = "Cajón: ${lugar.codigo}", fontWeight = FontWeight.Bold, fontSize = 18.sp)
                    Text(text = "Tarifa: $$${lugar.precioPorHora} / hr", fontSize = 14.sp)
                }
            }
        }
    }
}
```

```
Text(
  text = if (lugar.estaOcupado) "OCUPADO" else "DISPONIBLE",
  color = if (lugar.estaOcupado) MaterialTheme.colorScheme.error else
MaterialTheme.colorScheme.primary,
  fontWeight = FontWeight.Bold
)
}
}
}
}
}
```